The critical point in achieving resilience (or fault-tolerance) is keeping the faulty component isolated until the fault is fixed. By doing that, you'll prevent what Michael T. Nygard calls *crack propagation*. Think about Polar Bookshop. If Catalog Service enters a faulty state and becomes unresponsive, you don't want Order Service to be affected as well. Integration points between application services should be carefully guarded and made resilient to failures affecting the other party.

There are several patterns for building resilient applications. In the Java ecosystem, a popular library for implementing such patterns was Hystrix, developed by Netflix, but as of 2018 it entered maintenance mode and will not be developed further. Resilience4J gained a lot of popularity, filling the void left by Hystrix. Project Reactor, the Reactive Spring stack foundation, also provides some useful features for resilience.

In this section, you'll make the integration point between Order Service and Catalog Service more robust, using Reactive Spring to configure timeouts, retries, and fallbacks. In the next chapter, you'll learn more about building resilient applications with Resilience4J and Spring Cloud Circuit Breaker.

8.4.1 Timeouts

Whenever your application calls a remote service, you don't know if and when a response will be received. Timeouts (also called *time limiters*) are a simple, yet effective, tool for preserving the responsiveness of your application in case a response is not received within a reasonable time period.

There are two main reasons for setting up timeouts: